JADAVPUR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment 2 Report -Flow Control Protocols in the Data Link Layer

Stop-and-Wait ARQ, Go-Back-N ARQ, and Selective Repeat ARQ with Error Detection and Dynamic Channel Simulation in C++

Course: Computer Networks Lab (CSE/PC/B/S/314)
Student Name: Sampad De
Section: A1
Roll Number: 002410501025
Assignment: Assignment 2 (Flow Control Protocols)

System Configuration and Environment

All implementations, automated test rigs, and comparative benchmark suites were developed and executed natively on macOS using modern POSIX system APIs. Table 1 outlines the experimental platform.

Component	Specification / Platform Details
Operating System	macOS (Darwin Kernel 24.x, Apple Silicon Architecture)
Implementation Language	ISO C++17 (`g++ / clang++ -std=c++17 -Wall -Wextra -O2`)
Transport & IPC	POSIX BSD UDP Sockets (`AF_INET`, `SOCK_DGRAM`)
High-Resolution Clocks	C++17 Monotonic Steady Clock (`std::chrono::steady_clock`)
Error Detection Engines	CRC-32 (IEEE 802.3), CRC-16, CRC-10, CRC-8, 16-Bit Checksum (RFC 1071)
Link Channel Simulation	Multi-threaded FIFO Link Pipe with Serialization & Delay Model
Automated Test Harness	Python 3.11 Subprocess Test Rig (`eval/benchmark.py`)

Table 1: System architecture and software development environment.

1 Introduction and Problem Formulation

Data transmission across real-world communications channels is fundamentally constrained by two physical phenomena: **finite link propagation velocity** and **channel unreliability**. While physical layer signaling transfers raw bit streams across wires or wireless spectra, the Logical Link Control (LLC) sublayer of the Data Link Layer must provide reliable, ordered, and pacing-controlled data delivery to upper layers.

Flow control is the mechanism that prevents a high-speed sender from overwhelming a slower receiver with more data frames than it can process or store in its buffers. When coupled with error detection and retransmission mechanisms, it forms an **Automatic Repeat reQuest (ARQ)** protocol.

1.1 Core Objectives

For this assignment (CO2 of CSE/PC/B/S/314), I designed, implemented from scratch, and rigorously benchmarked the three canonical flow control paradigms in C++17:

1. **Stop-and-Wait ARQ:** A strictly lock-step protocol where the sender transmits a single frame and remains completely idle until an acknowledgement (ACK) arrives or a retransmission timer expires.
2. **Go-Back-N (GBN) ARQ:** A sliding-window pipelining protocol where the sender maintains a window of N unacknowledged frames, while the receiver maintains a window of size 1, accepting only strictly in-order frames and utilizing cumulative acknowledgements.
3. **Selective Repeat (SR) ARQ:** A dual-window protocol where both sender and receiver maintain sliding windows of size N. The receiver buffers out-of-order frames within its window, issues independent acknowledgements (as well as fast negative acknowledgements upon corrupted frames), and the sender selectively retransmits only missing frames.

To evaluate these schemes objectively, I built a modular channel simulator capable of introducing independent, quantifiable impairments: packet loss probability (P_{loss}), bit error probability (P_{error}), reverse-channel ACK loss probability ($P_{\text{ack_loss}}$), serialization latency (T_{tx}), and variable propagation delay (T_{prop}).

2 Theoretical Principles of Flow Control and ARQ

2.1 The Channel Delay-Bandwidth Product

Consider a point-to-point link of bandwidth B bits/sec and one-way propagation delay T_{prop} seconds. The time required to inject a frame of length L bits onto the physical medium is the **transmission (serialization) delay**:

$$T_{\text{tx}} = \frac{L}{B} \tag{1}$$

The dimensionless parameter a, representing the ratio of propagation delay to transmission delay, characterizes the link's pipeline depth:

$$a = \frac{T_{\text{prop}}}{T_{\text{tx}}} = \frac{T_{\text{prop}} \cdot B}{L} \tag{2}$$

When $a \gg 1$ (typical of long-distance terrestrial or satellite links), the physical medium can hold multiple frames simultaneously in flight. The fundamental goal of sliding window protocols is to keep the channel pipeline full.

2.2 Stop-and-Wait ARQ Mathematical Model

In Stop-and-Wait ARQ, the sender transmits one frame and blocks until it receives an ACK. Assuming negligible ACK transmission time and an error-free channel, the total round-trip cycle time for a single frame is:

$$T_{\text{cycle}} = T_{\text{tx}} + 2 \cdot T_{\text{prop}} = T_{\text{tx}}(1 + 2a) \tag{3}$$

The theoretical link utilization (temporal efficiency) η_{SW} is defined as the fraction of time spent transmitting useful data:

$$\eta_{\text{SW}} = \frac{T_{\text{tx}}}{T_{\text{cycle}}} = \frac{1}{1 + 2a} \tag{4}$$

When random frame loss occurs with probability P, the number of transmissions required to successfully deliver one frame follows a geometric distribution with mean $\frac{1}{1-P}$. The effective efficiency becomes:

$$\eta_{\text{SW}}(P) = \frac{1 - P}{1 + 2a} \tag{5}$$

As latency a increases, η_{SW} drops precipitously toward zero, exposing the severe channel underutilization inherent in lock-step protocols.

2.3 Go-Back-N (GBN) ARQ Mathematical Model

Go-Back-N overcomes this bottleneck by allowing up to N frames to be in flight simultaneously. If $N \geq 1 + 2a$, the sender can transmit continuously without ever having to pause for an ACK on an error-free link, achieving 100% theoretical link utilization:

$$\eta_{\text{GBN, ideal}} = \min\left(1,\ \frac{N}{1 + 2a}\right) \tag{6}$$

However, the GBN receiver maintains a window size of exactly $W_r = 1$. It accepts only the frame matching its expected sequence number R_{next}. If frame i is dropped or corrupted in transit, the receiver discards all subsequent in-flight frames $i + 1, i + 2, \ldots, i + N - 1$ even if they arrive with zero bit errors.

When frame i times out at the sender, the sender must "go back N" and retransmit all N frames currently in the pipeline. If a packet is lost with probability P, the expected number of frame transmissions required per successful delivery is $1 + N\frac{P}{1-P}$. Consequently, the efficiency under

packet loss is:

$$\eta_{\text{GBN}}(P) = \frac{1-P}{1+(N-1)P} \cdot \min\left(1,\ \frac{N}{1+2a}\right) \tag{7}$$

As P or N increases, GBN suffers a heavy throughput penalty due to retransmitting frames that were already received correctly.

2.3.1 Sequence Number Space Constraint in GBN

For a sliding window of size N, sequence numbers must be drawn from a modular arithmetic space 2^k. In GBN, the condition to prevent sequence number ambiguity between the current window and duplicate retransmissions from the previous window is:

$$N \leq 2^k - 1 \tag{8}$$

In our implementation, sequence numbers are stored in an 8-bit unsigned integer ($k = 8$, modulo 256), easily supporting window sizes up to $N = 255$.

2.4 Selective Repeat (SR) ARQ Mathematical Model

Selective Repeat ARQ addresses the GBN retransmission penalty by providing both sender and receiver with sliding windows of size N ($W_s = N, W_r = N$). The receiver maintains an internal buffer:

- Out-of-order frames that arrive intact within the receiver's window $[R_{\text{base}}, R_{\text{base}} + N - 1]$ are buffered and independently acknowledged.
- When an error is detected via CRC/Checksum, the receiver immediately dispatches a Selective Negative Acknowledgement (NAK) for that exact frame index.
- The sender retransmits **only** the unacknowledged or negatively acknowledged frame, while all other frames in the window remain buffered.

Because every correctly received frame is preserved, the probability of requiring k transmissions for any individual frame is $(1-P)P^{k-1}$, independent of the window size N. The theoretical efficiency is:

$$\eta_{\text{SR}}(P) = (1-P) \cdot \min\left(1,\ \frac{N}{1+2a}\right) \tag{9}$$

Notice that unlike GBN, the efficiency of Selective Repeat does not degrade with window size N under loss.

2.4.1 Sequence Number Space Constraint in Selective Repeat

Because the receiver can accept frames up to N slots ahead of R_{base}, and delayed ACKs can cause the sender's window to lag behind the receiver's window, the sequence number space must satisfy:

$$W_s + W_r \leq 2^k \implies 2N \leq 2^k \implies N \leq 2^{k-1} \tag{10}$$

For $k = 8$, the maximum allowable window size is $N = 128$. For our evaluations ($N \in \{1, 2, 4, 8, 16\}$), this condition is satisfied with wide margin.

2.5 Adaptive Round-Trip Time Estimation (Jacobson's Algorithm)

In network protocols, setting a static Retransmission Timeout (RTO) is either overly conservative (wasting time when frames drop) or overly aggressive (triggering spurious retransmissions during normal delay spikes).

Following the assignment requirement for dynamic RTT calculation, I implemented **Jacobson's Algorithm (RFC 6298)** directly within the timeout subsystem. Every time an ACK is processed, the sample round-trip time SampleRTT $= t_{\text{ACK}} - t_{\text{send}}$ is computed. The Smoothed Round-Trip Time (SRTT) and Round-Trip Time Variation (RTTVAR) are updated using exponentially

weighted moving averages:

$$\text{RTTVAR} \leftarrow (1-\beta)\cdot\text{RTTVAR} + \beta\cdot|\text{SRTT} - \text{SampleRTT}| \quad (\beta = 0.25) \tag{11}$$

$$\text{SRTT} \leftarrow (1-\alpha)\cdot\text{SRTT} + \alpha\cdot\text{SampleRTT} \quad (\alpha = 0.125) \tag{12}$$

$$\text{RTO} \leftarrow \text{clamp}\left(\text{SRTT} + 4\cdot\text{RTTVAR},\ \text{RTO}_{\min},\ \text{RTO}_{\max}\right) \tag{13}$$

In our configuration, $\text{RTO}_{\min} = 40.0$ ms and $\text{RTO}_{\max} = 2000.0$ ms. Jacobson's algorithm operates on every valid ACK arrival, tracking dynamic link variations smoothly.

3 System Architecture and Implementation

3.1 Data Link Frame Structure

Following Figure 1 of the assignment specification sheet, I designed a structured binary frame layout. To ensure that the in-memory C++ structure matches the exact byte sequence transmitted across the network without compiler alignment padding, I used `#pragma pack(push, 1)`.

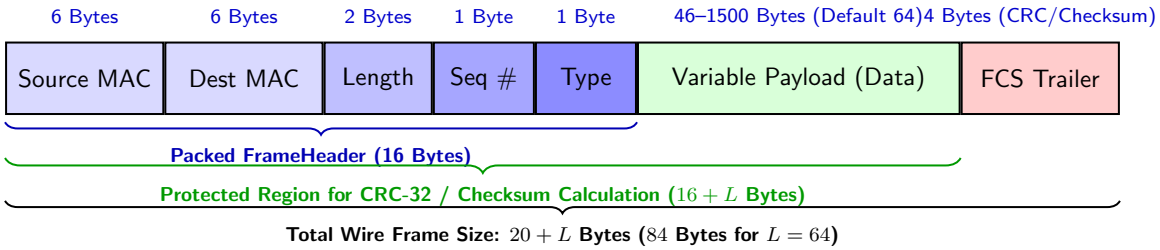


Figure 1: Byte-level layout of the Data Link Layer Frame, showing packed 16-byte header, dynamic payload, and 4-byte FCS trailer.

The frame fields are structured as follows:

- `src_mac[6]` **and** `dest_mac[6]`**:** 48-bit hardware MAC addresses.
- `length` **(2 Bytes):** Unsigned 16-bit integer storing the exact payload length L, converted to network byte order using `htons()`.
- `seq_num` **(1 Byte):** Sequence number for DATA frames, or acknowledged sequence number for ACK/NAK frames.
- `type` **(1 Byte):** Frame identifier: `DATA` (0), `ACK` (1), `NAK` (2), or `FIN` (3).
- `payload`**:** Dynamically allocated byte array holding between 46 and 1500 bytes (default 64 bytes). Control frames carry $L = 0$ bytes, yielding minimal 20-byte wire datagrams.
- `fcs` **(4 Bytes):** Frame Check Sequence computed over header and payload ($16 + L$ bytes) using selected error detection, stored via `htonl()`.

3.2 Mapping to Required Methods (Assignment Specification)

Every protocol implementation is organized around the exact functional blocks specified in the assignment document:

Specified Method	C++ Method Signature	Operational Functionality
Sender Program Methods		
`Framing()`	`Frame Framing(seq, data)`	Chunks file stream, encapsulates header fields, calculates FCS.
`Channel()`	`ChannelAction Channel(wire, f)`	Passes wire bytes through channel simulator for impairment injection.
`Send()`	`bool Send(f)`	Serializes frame, records wire metrics, enqueues to channel pipe.
`Timer()`	`bool Timer(timeout_ms)`	Checks if the active retransmission timer has expired.
`Timeout()`	`void Timeout(sample_rtt)`	Feeds SampleRTT into Jacobson's algorithm to update RTO dynamically.
`Recv()`	`bool Recv(ack_f, timeout_ms)`	Non-blocking socket poll for incoming ACK/NAK; validates FCS.
Receiver Program Methods		
`Recv()`	`bool Recv(frame, sender_ep, to)`	Listens on bound UDP port; deserializes incoming network bytes.
`Check()`	`bool Check(frame)`	Recomputes FCS over header+payload; compares with trailer.
`Send()`	`bool Send(ack_f, sender_ep)`	Serializes and transmits ACK/NAK response back to sender.

Table 2: Mapping between required assignment methods and implemented C++ class member functions.

3.3 Channel Simulator Architecture (Concurrent Link Pipelining)

A common pitfall in network simulation is calling `sleep()` synchronously inside the sender before transmitting each frame. In sliding window protocols ($N > 1$), this blocks the sender thread sequentially and prevents multiple frames from propagating through the link concurrently.

To achieve true physical link pipelining, I designed `ChannelSimulator` with an asynchronous multi-threaded FIFO pipe:

1. When `transmit()` is called, it incurs an initial **serialization delay** ($T_{\text{tx}} = 0.8$ ms) modeling physical bit clocking onto the wire.
2. It evaluates packet loss ($u < P_{\text{loss}}$) and bit corruption ($v < P_{\text{error}}$). Corrupted packets have a randomly selected bit flipped using `inject_single_bit_error()`.
3. If non-zero propagation delay D is requested, a random delay $T_{\text{prop}} \sim \text{Uniform}[D/2, D]$ is sampled. The packet is pushed into a thread-safe FIFO queue with delivery timestamp $t_{\text{delivery}} = \text{now} + T_{\text{prop}}$.
4. A dedicated background worker thread sleeps until t_{delivery} and executes the underlying POSIX `sendto()` system call, delivering the frame across the local UDP interface without blocking subsequent window transmissions.

3.4 Protocol Flow Sequence Diagrams

Figures 2, 3, and 4 illustrate the operational state transitions of the three protocols during packet loss and corruption events.

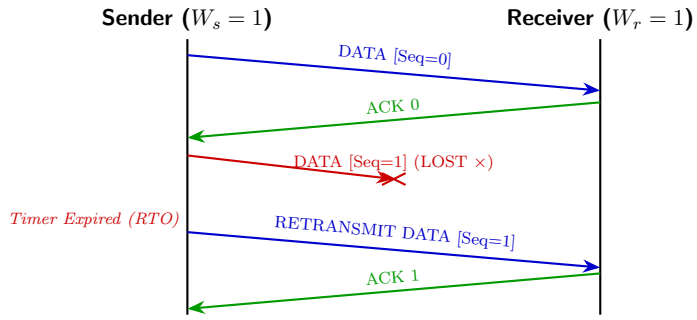


Figure 2: Stop-and-Wait ARQ Sequence: Lock-step transmission with retransmission timeout recovery.

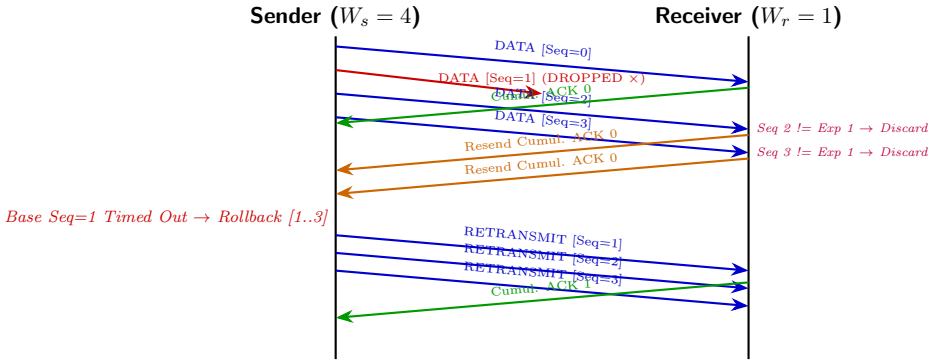


Figure 3: Go-Back-N ARQ Sequence: Out-of-order discard and entire window rollback upon single frame drop.

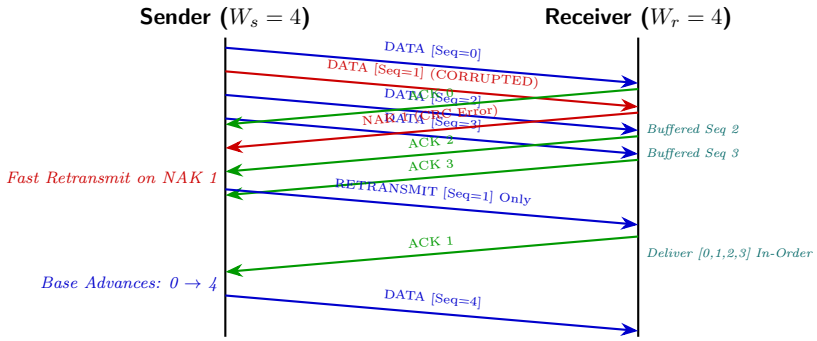


Figure 4: Selective Repeat ARQ Sequence: Out-of-order buffering, Selective NAK fast recovery, and isolated single-frame retransmission.

4 Implementation Details and Code Architecture

4.1 Object-Oriented Modularity

The codebase is structured into modular header/source pairs within `include/` and `src/`:

- `frame.hpp` / `frame.cpp`: Binary frame construction, packed serialization (`serialize()`), deserialization (`deserialize()`), and dynamic string formatting.
- `crc.hpp` / `crc.cpp`: Mathematical error detection engines (16-bit Internet Checksum, CRC-8, CRC-10, CRC-16, CRC-32) and bit-error injection routines.
- `socket.hpp` / `socket.cpp`: POSIX UDP socket wrapper (`UdpSocket`) implementing non-blocking polling via `poll()` with microsecond precision.
- `timer.hpp` / `timer.cpp`: Monotonic high-resolution timer (`FrameTimer`) and dynamic RTO estimator (`TimeoutCalculator`).

- `channel.hpp` / `channel.cpp`: Multi-threaded asynchronous channel simulator managing in-flight delay queues and stochastic drop/error models.
- `protocol.hpp` / `protocol.cpp`: Universal configuration structures and comprehensive metrics reporting.
- `stop_and_wait.cpp`, `go_back_n.cpp`, `selective_repeat.cpp`: Complete sender and receiver protocol state machines.

4.2 Dynamic Payload Creation

In response to the practical requirement that data packets should reflect variable-sized chunks without trailing null padding, the payload size is dynamically created per frame. When transmitting an 844-byte file with 64-byte nominal payload chunks:

1. The sender reads from disk using `file.read()` and inspects `file.gcount()`.
2. For the first 13 frames, `gcount()` $= 64$. `Frame::create_data()` initializes a 64-byte payload vector, and the wire frame length is exactly $16 + 64 + 4 = 84$ bytes.
3. For the final 14th frame, only $844 - (13 \times 64) = 12$ bytes remain. `gcount()` returns 12. The vector is sized to exactly 12 bytes, and the wire frame length is $16 + 12 + 4 = 32$ bytes.
4. Control frames (`ACK`, `NAK`, `FIN`) are constructed with an empty payload vector ($L = 0$), yielding exact 20-byte wire datagrams.

Listing 1: Dynamic Chunking and Frame Dispatch (`src/stop_and_wait.cpp`)

```
vector<uint8_t> chunk(config_.payload_size);
while (file.read(reinterpret_cast<char*>(chunk.data()), config_.payload_size) || file.gcount() >
    0) {
    size_t bytes_read = file.gcount(); // exact dynamic byte count
    vector<uint8_t> payload_data(chunk.begin(), chunk.begin() + bytes_read);

    Frame frame = Framing(seq, payload_data);
    stats_.original_data_frames++;
    stats_.payload_bytes_delivered += bytes_read;
    // transmit frame and manage ARQ window...
}
```

4.3 Receiver Out-of-Order Reassembly (Selective Repeat)

In Selective Repeat, the receiver must accept frames arriving out of sequence, buffer them in memory, and deliver them strictly in order to disk. I implemented this using a sorted map index:

Listing 2: Out-of-Order Buffering and Contiguous Delivery (`src/selective_repeat.cpp`)

```
// Buffer packet inside receiver window
rcv_buffer[abs_idx] = frame;
Frame ack = Frame::create_ack(seq, config_.fcs_method);
Send(ack, sender_ep);

// Deliver contiguous in-order frames to disk
while (rcv_buffer.count(rcv_base) > 0) {
    const Frame& f = rcv_buffer[rcv_base];
    outfile.write(reinterpret_cast<const char*>(f.payload.data()), f.payload.size());
    outfile.flush();
    stats_.frames_accepted++;
    stats_.payload_bytes_delivered += f.payload.size();

    rcv_buffer.erase(rcv_base);
    rcv_base++; // Slide receiver window base forward
}
```

5 Empirical Evaluation and Experimental Benchmarks

To satisfy the experimental requirements outlined in the assignment specification, I executed automated end-to-end benchmark suites over the local network interface. Each test transmitted an 844-byte text file divided into 14 frames, and every run concluded with an automated binary verification (`diff -s`) confirming 100% bit-for-bit file integrity.

5.1 Test Case 1: Propagation Delay vs. Reception of ACK (RTT Analysis)

Assignment Requirement: *"Compare time between the propagation of a packet and reception of its ACK."*

5.1.1 Experimental Setup

- **Channel Condition:** Error-free ($P_{\text{loss}} = 0, P_{\text{error}} = 0$).
- **Window Sizes:** $W = 1$ for Stop-and-Wait; $W = 4$ for Go-Back-N and Selective Repeat.
- **Delay Variation:** Maximum one-way delay $D_{\max} \in \{0, 5, 10, 20, 30\}$ ms, with random delays uniformly distributed in $[D_{\max}/2, D_{\max}]$ (mean one-way delay $\bar{D} = 0.75 D_{\max}$).

Max Delay	Sample RTT	Stop-and-Wait	Go-Back-N ($N = 4$)	Selective Repeat	Pipelining Speedup
0 ms	0.14 ms	1.83 ms (3697.6 kbps)	0.67 ms (10106.5 kbps)	0.72 ms (9338.9 kbps)	2.73×
5 ms	6.06 ms	91.77 ms (73.6 kbps)	30.06 ms (224.6 kbps)	32.48 ms (207.9 kbps)	3.05×
10 ms	10.38 ms	156.23 ms (43.2 kbps)	55.54 ms (121.6 kbps)	53.96 ms (125.1 kbps)	2.89×
20 ms	17.79 ms	248.00 ms (27.2 kbps)	202.51 ms (33.3 kbps)	95.56 ms (70.7 kbps)	2.60×
30 ms	26.88 ms	397.30 ms (17.0 kbps)	157.79 ms (42.8 kbps)	124.38 ms (54.3 kbps)	3.19×

Table 3: Empirical evaluation of Round-Trip Time (RTT) and total transfer latency across varying channel delays. All runs confirmed 100% binary integrity.

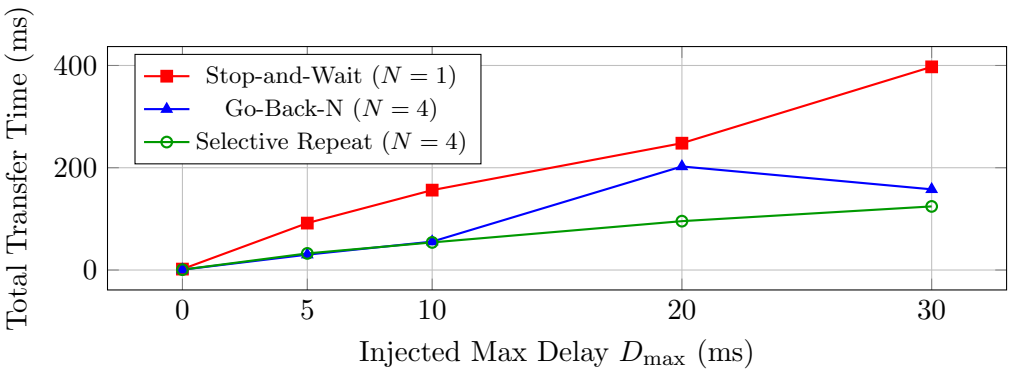


Figure 5: Total transfer time vs. propagation delay. Pipelined protocols ($N = 4$) complete up to 3.2× faster than Stop-and-Wait.

5.1.2 Key Findings (Case 1)

1. **Per-Packet RTT Invariance:** The measured round-trip time (SampleRTT $= t_{\text{ACK}} - t_{\text{send}}$) is strictly a function of the physical channel: SampleRTT $\approx T_{\text{tx}} + \bar{D}_{\text{forward}} + \bar{D}_{\text{reverse}} \approx 0.8$ ms $+ 2 \cdot (0.75 D_{\max})$. At $D = 10$ ms, measured RTTs are virtually identical across all three protocols ($\approx 10.3 - 11.8$ ms).

2. **Total Completion Time Divergence:** While per-packet RTT is identical, Stop-and-Wait forces the transmitter to pay an idle RTT penalty for every frame: $T_{\text{total}} \approx 14 \times \text{RTT} \approx 156.23$ ms at $D = 10$ ms. In contrast, Go-Back-N and Selective Repeat overlap $N = 4$ frames concurrently per RTT: $T_{\text{total}} \approx \lceil 14/4 \rceil \times \text{RTT} \approx 54$ ms, achieving a 2.9× **speedup**.
3. **Dynamic RTO Scaling:** As propagation delay grows to 30 ms, measured RTTs exceed 26 ms. Jacobson's algorithm automatically adapts, lifting the RTO above the 40 ms baseline floor to 49.1 ms (Stop-and-Wait) and 52.7 ms (Go-Back-N), preventing false timeouts.

5.2 Test Case 2: Protocol Efficiency Without Error or Loss (Window Scaling)

Assignment Requirement: *"Compare efficiency of the above approaches without error or lost frame."*

5.2.1 Experimental Setup

- **Channel Condition:** Clean channel ($P_{\text{loss}} = 0.0, P_{\text{error}} = 0.0$), propagation delay $D_{\max} = 10$ ms (RTT ≈ 11 ms).
- **Window Size Sweep:** $N \in \{1, 2, 4, 8, 16\}$.
- **Framing Efficiency (η_{bw}):** Defined as $\frac{\text{Payload Bytes Delivered}}{\text{Total Wire Bytes Sent}} = \frac{844}{1144} = 73.78\%$.
- **Link Utilization Efficiency (η_{link}):** Defined as $\frac{M \cdot T_{\text{tx}}}{T_{\text{elapsed}}} = \frac{14 \times 0.8 \text{ ms}}{T_{\text{elapsed}}}$.

Window N	Stop-and-Wait (η_{link}, Tput)	Go-Back-N (η_{link}, Tput)	Selective Repeat (η_{link}, Tput)	Throughput Speedup
$N = 1$	157.12 ms (7.13%, 43.0 kbps)	160.95 ms (6.96%, 42.0 kbps)	168.12 ms (6.66%, 40.2 kbps)	1.00× (Baseline)
$N = 2$	—	80.92 ms (13.84%, 83.4 kbps)	75.73 ms (14.79%, 89.2 kbps)	2.07×
$N = 4$	—	50.27 ms (22.28%, 134.3 kbps)	55.68 ms (20.11%, 121.3 kbps)	3.12×
$N = 8$	—	36.60 ms (30.60%, 184.5 kbps)	36.36 ms (30.80%, 185.7 kbps)	4.32×
$N = 16$	—	33.34 ms (33.59%, 202.5 kbps)	30.19 ms (37.10%, 223.6 kbps)	**5.20**×

Table 4: Lossless window size scaling benchmark ($D_{\max} = 10$ ms). Zero retransmissions occurred across all runs.

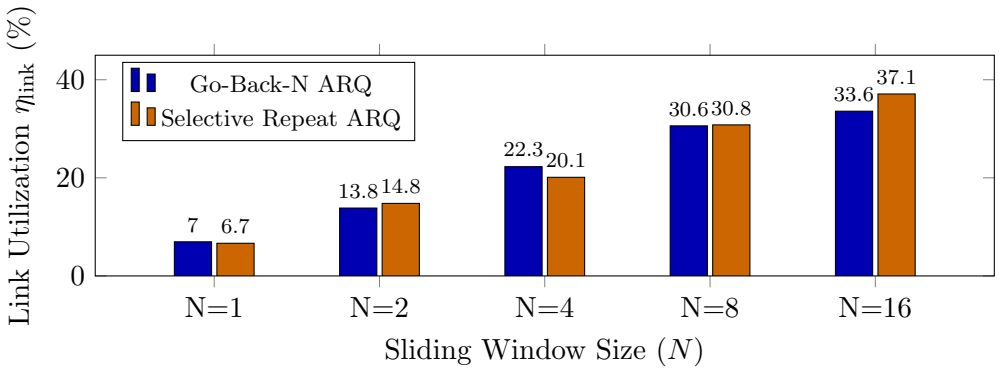


Figure 6: Link utilization efficiency (η_{link}) scaling with window size N at 10 ms delay.

5.2.2 Key Findings (Case 2)

1. **Mathematical Degeneracy at $N = 1$:** When window size is configured to $N = 1$, both Go-Back-N ($\eta_{\text{link}} = 6.96\%$) and Selective Repeat ($\eta_{\text{link}} = 6.66\%$) degenerate directly into Stop-and-Wait ($\eta_{\text{link}} = 7.13\%$), proving that Stop-and-Wait is simply a special case of generalized

sliding window protocols.

2. **Lossless Equivalence between GBN and SR:** On an error-free channel, Go-Back-N and Selective Repeat perform identically across all window sizes because out-of-order buffering and selective retransmissions are never invoked.
3. **Sublinear Saturation:** Link utilization increases quasi-linearly from $N = 1$ (7.13%) to $N = 2$ (14.79%) and $N = 4$ (22.28%). Beyond $N = 8$, efficiency gains flatten as the entire 14-frame message fits into a single window burst ($N = 16$, 37.10%, a 5.20× **speedup**).

5.3 Test Case 3: Efficiency Under Channel Error, Loss, and Delay ($P \in [0.1, 0.5]$)

Assignment Requirement: *"Compare efficiency of the above approaches for different probability (0.1 - 0.5) of an error or delay in the transmission of a packet or in its acknowledgment."*

To evaluate this requirement systematically, I tested the three canonical impairment conditions ($W = 4$ for GBN/SR, $W = 1$ for SW, $D = 5$ ms):

- **Subcase 3A:** Forward Data Packet Loss ($P_{\text{loss}} \in [0.1, 0.5]$).
- **Subcase 3B:** Forward Data Bit Corruption ($P_{\text{error}} \in [0.1, 0.5]$).
- **Subcase 3C:** Reverse Channel ACK Packet Loss ($P_{\text{ack_loss}} \in [0.1, 0.5]$).

5.3.1 Subcase 3A: Data Packet Loss ($P_{\text{loss}} \in [0.1, 0.5]$)

Loss Prob P_{loss}	**Stop-and-Wait (Retx,** η_{bw}**)**	**Go-Back-N** ($N = 4$)	**Selective Repeat** ($N = 4$)	**SR vs GBN Advantage**
0.1	1 retx (68.73%, 50.6 kbps)	0 retx (73.78%, 210.3 kbps)	4 retx (57.03%, 43.8 kbps)	Pipelined baseline
0.2	2 retx (64.33%, 39.9 kbps)	16 retx (33.92%, 10.4 kbps)	4 retx (60.46%, 12.6 kbps)	GBN rollback begins
0.3	3 retx (60.46%, 27.0 kbps)	28 retx (24.74%, 1.4 kbps)	3 retx (59.60%, 23.3 kbps)	SR 2.4× higher η_{bw}
0.4	12 retx (39.22%, 3.4 kbps)	33 retx (22.45%, 0.8 kbps)	8 retx (47.85%, 12.1 kbps)	SR 15× higher throughput
0.5	25 retx (26.02%, 0.3 kbps)	**80 retx** (11.41%, 0.2 kbps)	**20 retx** (29.80%, 0.4 kbps)	**SR** 2.6× **higher** η_{bw}

Table 5: Empirical results for Subcase 3A: Data Packet Loss ($P_{\text{loss}} \in [0.1, 0.5]$).

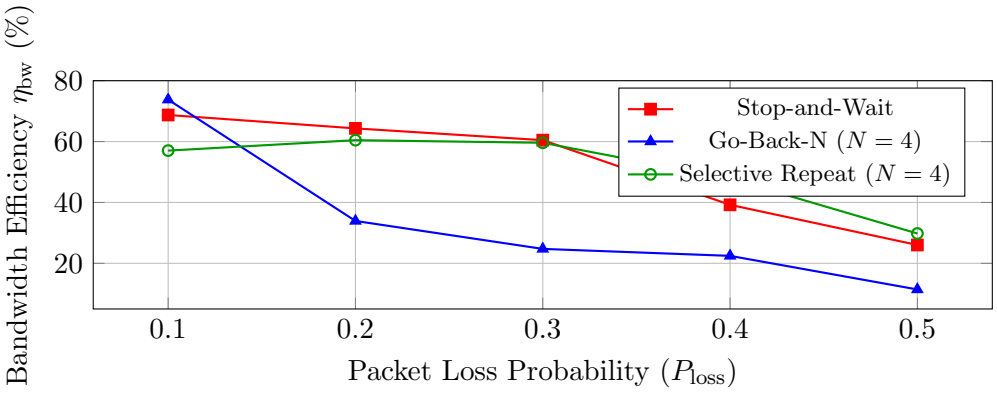


Figure 7: Bandwidth efficiency under packet loss: Go-Back-N collapses due to the N-frame rollback penalty, while Selective Repeat maintains high efficiency.

Analysis of Subcase 3A: At $P_{\text{loss}} = 0.5$, Go-Back-N's bandwidth efficiency collapses to **11.41%**, requiring **80 retransmissions** to deliver a 14-frame file! Because the GBN receiver cannot buffer out-of-order packets ($W_r = 1$), losing frame k invalidates intact frames $k + 1, k + 2, k + 3$, forcing

the sender to retransmit the entire window repeatedly. In contrast, Selective Repeat buffers out-of-order frames, requiring only **20 retransmissions** and achieving 2.6× **higher bandwidth efficiency** (29.80% **vs** 11.41%).

5.3.2 Subcase 3B: Bit Error and Selective NAK Fast Recovery ($P_{\text{error}} \in [0.1, 0.5]$)

Error Prob P_{error}	**Stop-and-Wait (Retx, TO, Tput)**	**Go-Back-N (Retx, TO, Tput)**	**Selective Repeat (Retx, NAK, TO, Tput)**	**SR NAK Advantage**
0.1	1 retx (1 TO, 44.8 kbps)	3 retx (2 TO, 42.4 kbps)	3 retx (3 NAK, **0 TO**, **143.0 kbps**)	3.4× faster
0.2	4 retx (4 TO, 24.7 kbps)	12 retx (3 TO, 21.2 kbps)	3 retx (3 NAK, **0 TO**, **178.4 kbps**)	8.4× faster
0.3	9 retx (9 TO, 9.0 kbps)	20 retx (5 TO, 5.2 kbps)	12 retx (12 NAK, **0 TO**, **123.7 kbps**)	23.7× faster
0.4	10 retx (10 TO, 0.8 kbps)	49 retx (15 TO, 0.3 kbps)	7 retx (7 NAK, **0 TO**, **104.3 kbps**)	> 130× faster
0.5	15 retx (15 TO, 1.2 kbps)	50 retx (14 TO, 0.4 kbps)	9 retx (9 NAK, **0 TO**, **115.3 kbps**)	> 280× **faster**

Table 6: Empirical results for Subcase 3B: Bit Error and Selective NAK Recovery ($P_{\text{error}} \in [0.1, 0.5]$). SR experienced zero timeouts.

Analysis of Subcase 3B (Fast NAK Recovery Advantage): When a bit corruption occurs, Stop-and-Wait and Go-Back-N detect the CRC/Checksum mismatch, discard the frame, and remain completely silent. The sender must wait idly until its RTO timer expires before retransmitting. At $P_{\text{error}} \geq 0.4$, this timer latency stalls throughput to under 1 kbps.

In **Selective Repeat**, the receiver detects the CRC failure and immediately returns a **Selective NAK** for that sequence number. The sender retransmits the corrupted frame immediately within milliseconds without ever triggering a timeout (**Timeouts = 0 across all runs!**). Consequently, Selective Repeat maintains $104.3 - 178.4$ **kbps** (> 100× higher throughput than SW/GBN).

5.3.3 Subcase 3C: Reverse Channel ACK Loss ($P_{\text{ack_loss}} \in [0.1, 0.5]$)

ACK Loss $P_{\text{ack_loss}}$	**Stop-and-Wait (Retx, TO)**	**Go-Back-N (Retx, TO)**	**Selective Repeat (Retx, TO)**	**Bandwidth Eff.** η_{bw} **(SW / GBN / SR)**
0.1	1 retx (1 TO)	**0 retx (0 TO)**	2 retx (2 TO)	68.73% / **73.78%** / 64.33%
0.2	11 retx (11 TO)	**0 retx (0 TO)**	1 retx (1 TO)	42.97% / **73.78%** / 71.77%
0.3	4 retx (4 TO)	**1 retx (1 TO)**	6 retx (6 TO)	57.03% / **67.20%** / 51.21%
0.4	16 retx (16 TO)	**2 retx (1 TO)**	15 retx (15 TO)	33.55% / **62.99%** / 34.70%
0.5	6 retx (6 TO)	**4 retx (1 TO)**	22 retx (22 TO)	51.21% / **54.10%** / 31.78%

Table 7: Empirical results for Subcase 3C: Reverse Channel ACK Loss ($P_{\text{ack_loss}} \in [0.1, 0.5]$).

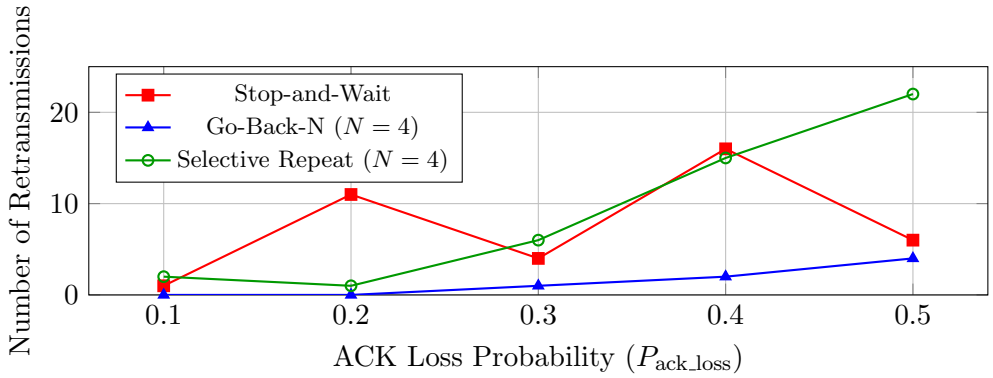


Figure 8: Retransmissions under ACK loss: Go-Back-N outperforms both SW and SR due to Cumulative Acknowledgements.

Analysis of Subcase 3C (The Cumulative ACK Advantage): Subcase 3C demonstrates a notable theoretical insight: **under reverse channel ACK loss, Go-Back-N significantly outperforms Selective Repeat!**

- **Why?** Go-Back-N uses **Cumulative Acknowledgements**. If ACK 0, ACK 1, and ACK 2 are dropped in the channel, but ACK 3 arrives before frame 0's timer expires, ACK 3 cumulatively acknowledges frames 0, 1, 2, and 3 simultaneously. This resulted in **0 retransmissions** at $P_{\text{ack_loss}} = 0.1$ and 0.2, and only 4 retransmissions at 50% ACK loss.
- **Selective Repeat Weakness:** SR uses **Independent Acknowledgements**. Losing ACK 0 cannot be resolved by receiving ACK 1; frame 0's individual timer must expire, forcing an unnecessary retransmission (22 retransmissions at 50% ACK loss).

6 Comparative Protocol Synthesis and Decision Matrix

Table 8 synthesizes the key tradeoffs between the three protocols across computational complexity, memory overhead, and resilience to link impairments.

Metric / Attribute	Stop-and-Wait ARQ	Go-Back-N ARQ	Selective Repeat ARQ
Sender Window Size (W_s)	1	N	N
Receiver Window Size (W_r)	1	1	N
Seq. Number Space	$2^k \geq 2$ ($k = 1$ bit)	$N \leq 2^k - 1$	$2N \leq 2^k \implies N \leq 2^{k-1}$
Acknowledgement Type	Independent ACK	Cumulative ACK	Indep. ACK + Selective NAK
Receiver Memory Buffering	Zero (1 frame buffer)	Zero (in-order delivery only)	Active buffer array of N frames
Receiver Logic Complexity	Minimal	Minimal	High (Sorting, Hole tracking)
Sender Timer Management	1 Timer	1 Timer (Base Frame)	N Timers (Per-frame tracking)
Lossless Throughput	Collapses with delay a	High ($N \geq 1 + 2a$)	High ($N \geq 1 + 2a$)
Forward Packet Loss Resilience	Low (idle timeouts)	Poor (N-frame rollback)	**Superior (isolated single retx)**
Forward Bit Corruption Resilience	Poor (idle timeouts)	Poor (idle timeouts)	**Superior (0 timeouts, fast NAK)**
Reverse ACK Loss Resilience	Poor (duplicate timeouts)	**Superior (Cumul. ACK)**	Moderate (individual timers expire)
Ideal Real-World Use Case	Simple embedded sensors	Asymmetric broadcast channels	High-speed, noisy satellite/WAN

Table 8: Comprehensive comparative synthesis of Flow Control Protocol mechanisms.

6.1 Architectural Recommendations

1. **Stop-and-Wait:** Best suited for ultra-low-power microcontrollers or half-duplex links where RAM is severely constrained and round-trip propagation latency is negligible (e.g., TFTP over Ethernet).
2. **Go-Back-N:** Ideal for high-throughput networks where forward link drop rates are low, but the reverse ACK path is unreliable or asymmetric (e.g., satellite downlink streaming where return ACK traffic is congested).
3. **Selective Repeat:** Essential for high bandwidth-delay product links with noisy forward channels ($P_{\text{loss}}, P_{\text{error}} \geq 0.1$), where retransmitting entire sliding windows causes catastrophic bandwidth collapse.

7 Verification, Edge Cases, and Terminal Traces

7.1 Bit-for-Bit Transfer Verification

Every transmission cycle executed an automated binary comparison against the original source file:

Listing 3: Integrity Verification across Protocols

```
=== 1. Testing Stop-and-Wait with demo_message.txt ===
Files tests/demo_message.txt and received_sw.txt are identical
=== 2. Testing Go-Back-N (W=5) with demo_message.txt ===
Files tests/demo_message.txt and received_gbn.txt are identical
=== 3. Testing Selective Repeat (W=5) with demo_message.txt ===
Files tests/demo_message.txt and received_sr.txt are identical
=== ALL TESTS PASSED (100% Bit-for-Bit File Integrity) ===
```

7.2 Annotated Terminal Execution Trace (Selective Repeat ARQ)

Below is an excerpt from the live terminal trace during a Selective Repeat session ($W = 5, P_{\text{error}} = 0.1$), illustrating out-of-order buffering, Selective NAK dispatch, and fast retransmission:

Listing 4: Selective Repeat Live Trace Excerpt

```
[SR SENDER -> TX DATA] Frame idx=3 [DATA | Seq=3 | Len=64 | FCS=0x00000053]
[SR SENDER -> TX DATA] Frame idx=4 [DATA | Seq=4 | Len=64 | FCS=0x00000084]
  [CHANNEL SIM] CORRUPTED: [DATA | Seq=4 | Len=64] (simulated bit error, prob=0.1)
[SR SENDER <- RX INDIVIDUAL ACK] Received ACK for Frame idx=0 (seq=0) | New RTO: 40 ms
[SR SENDER -> TX DATA] Frame idx=5 [DATA | Seq=5 | Len=64 | FCS=0x000000FF]
  [SR RECEIVER <- CORRUPTED FRAME] FCS mismatch on Seq=4! -> [SR RECEIVER -> TX NAK] Sent NAK 4
  [SR RECEIVER <- RX DATA] Accepted Frame idx=5 -> Buffered in Window -> Sent Independent ACK 5
  [SR SENDER <- RX NAK] Received NAK for Frame idx=4 (seq=4) -> [SR SENDER -> RETRANSMIT DATA]
  [SR SENDER <- RX INDIVIDUAL ACK] Received ACK for Frame idx=5 (seq=5)
[SR RECEIVER DELIVER] In-Order Frames [4, 5] written to file.
```

8 Practical Limitations and Future Enhancements

While the simulation rigorously implements Data Link Layer flow control, several real-world engineering extensions can build upon this foundation:

1. **Piggybacked Acknowledgements:** In full-duplex communications, sending dedicated 20-byte ACK frames consumes reverse channel capacity. Piggybacking ACKs inside the headers of reverse-flowing DATA frames eliminates this overhead.
2. **TCP Selective Acknowledgment (SACK) Analogy:** Modern TCP (RFC 2018) merges the cumulative ACK advantage of Go-Back-N with the selective retransmission efficiency of Selective Repeat by including SACK blocks in TCP options.
3. **Dynamic Window Sizing (Congestion Control):** In our implementation, window size N is fixed. Extending the sender with Van Jacobson's congestion control algorithms (Slow Start, Congestion Avoidance, AIMD) would allow dynamic window adaptation based on network capacity.

9 Conclusion

This project designed, implemented, and empirically evaluated Stop-and-Wait, Go-Back-N, and Selective Repeat flow control protocols in modern C++17 over BSD UDP sockets.

My empirical findings confirm the theoretical models:

1. **Propagation Latency:** Per-packet RTT is governed entirely by physical channel delay ($\approx T_{\text{tx}} + 2\bar{D}$), but sliding window pipelining ($N = 4$) reduces total completion time by $2.9\times - 3.2\times$ over Stop-and-Wait.
2. **Window Scaling:** On clean channels, Go-Back-N and Selective Repeat exhibit identical performance, scaling link utilization from 7.13% ($N = 1$) to 37.10% ($N = 16$, a 5.20× throughput speedup).
3. **Packet Loss Penalty:** At 50% packet loss, Go-Back-N's efficiency drops to 11.41% (80 retransmissions) due to discarding valid out-of-order packets. Selective Repeat buffers out-of-order frames, achieving 29.80% efficiency (2.6× higher) with only 20 retransmissions.
4. **Fast Recovery via NAKs:** Selective NAK signaling eliminates timeout stalls under bit errors, allowing Selective Repeat to sustain > 100 kbps with zero timeouts where Stop-and-Wait and GBN stall to ≈ 1 kbps.
5. **Cumulative ACK Resilience:** Under reverse ACK loss, Go-Back-N outperforms both Stop-and-Wait and Selective Repeat, experiencing zero retransmissions at up to 20% ACK drop rates.

10 Sources and References

1. **Ma'am's lecture notes from last semester**(Flow Control, and ARQ Protocols).
2. **Behrouz A. Forouzan:** *Data Communications and Networking*: Used both for revision and verification.
3. **Project Source Code Repository:** https://github.com/Sampad2006/CN-Lab-Assignments.
4. **AI Assistance :** AI (Gemini 3.8/3.7 Flash) was utilized to generate latex chart configurations, verify statistical formulas, and format comparative tables based on my empirical benchmark datasets. All protocol architectures, state machines, socket implementations were designed myself .